UNIVERSITY OF WISCONSIN–MADISON · CS 571 BUILDING USER INTERFACES

Midterm Exam Reference Sheet

Lectures 1–13
Fall 2026

01 Introduction

- **JSON**: a human-readable, language-agnostic text format for structured data. Types: string, number, boolean, array, object, `null`; values can nest.

02 Web Dev Basics 1

HTML · CSS · JS

- **Including CSS/JS:** inline, internal (`<style>` / `<script>`), or external (`<link>` / `<script src>`). **Prefer external.**
- **Cascade:** the style closest to the element wins; `!important` overrides.
- `var`: function scope · `let` / `const`: block scope (`const` can't be re-assigned). Function *definitions* are hoisted; expressions and arrow functions are not.
- **Falsy:** `false` `undefined` `null` `0` `NaN` `""`

03 Web Dev Basics 2

JSON · APIS · FETCH

- **JS object** = executable code; **JSON** = language-agnostic representation (`"quoted"` keys, no comments, no trailing commas).
- `JSON.parse`: string → object · `JSON.stringify`: object → string.
- **API:** definitions and protocols for communication via serialization and de-serialization of objects.
- **Promise:** the eventual completion or failure of an async operation. `fetch` is async and non-blocking, so code after it runs *before* its `.then`.
- **Callback:** a function passed as an argument and invoked inside the outer function.

04 Web Dev Basics 3

DECLARATIVE JS · BOOTSTRAP

- **Declarative** = *what* (preferred) · **imperative** = *how*.
- `filter(cb)` → new array of the elements where `cb` returns `true` · `map(cb)` → new array of each `cb` return value · `some` / `every` → boolean (any/all pass) · `reduce((prev, curr) => …, start)` → one value.
- `a ?? b` → `b` only if `a` is `null` / `undefined`. Spread `{...a}` = *shallow* copy; `JSON.parse(JSON.stringify(a))` = *deep* copy.
- **Bootstrap** categories: Layouts, Content, Components, Utilities. **Responsive design:** adapts to screen sizes via width *breakpoints*.
- `container` → `row` → `col-*` out of **12**. Breakpoints run small → large: `xs` `sm` `md` `lg` `xl` `xxl`. Each class applies at its breakpoint *and up*, so `col-12` `col-md-6` `col-lg-3` = 1, then 2, then 4 per row.

05 Design Thinking

DESIGN

UNDERSTAND | EXPLORE | MATERIALIZE

Empathize | Define | Ideate | Prototype | Test | Implement

- **Design thinking:** an *approach* (hands-on, user-centric problem solving) and a *process* (above).
- **Empathize:** learn what users *do, say, think, feel* (contextual inquiry, ethnography, diary studies, cultural probes).
- **Contextual inquiry:** go where the user works/lives → observe → talk. It is an inquiry, *not an interview*; the client is the *master*, you are the *apprentice*; don't distract from the task.
- **Define:** gather → analyze → recommend (insight).
- **Affinity diagramming** (affinity mapping, collaborative sorting, snowballing): cluster messy qualitative data by "affinity." Bottom-up: disorganized notes → group by likeness → consult team → re-organize → name each theme.
- **Ideation:** active, creative, exploratory, highly iterative, fast-moving collaborative idea generation. Consider workspace, team, process, rules of engagement, and how ideas are captured.
- **IDEO rules:** defer judgment · encourage wild ideas · build on others' ideas · stay focused · one conversation at a time · be visual · go for quantity.
- **Conceptual design:** an abstract characterization of the context, use, or experience of an envisioned solution, highlighting its main premise.
- **Sketches** are quick, cheap, disposable, plentiful, and annotated. A **storyboard** is a sequence of frames showing interaction with the system, capturing social, environmental, and technical factors.
- **Prototype:** real, tactile representations of ideas. **Test:** expert-based review or user-based testing. **Implement:** put the vision into effect.
- **Think-aloud:** representative users do representative tasks; observe and record what they say and do (obstacles, pain points, failures). It can be moderated/unmoderated, field/lab, and concurrent/retrospective.

06 Web Dev 1

REACT BASICS

- **Virtual DOM:** describe *what*; React updates the DOM. **Reconciliation:** diffing and syncing the virtual and real DOM.
- `react` = platform-agnostic reconciler · `react-dom` → browser · `react-native` → iOS/Android.
- Setting state is **async** (the old value persists until re-render). `setX(old => …)` updates from the previous value. Initial state applies on mount only.
- **HTTP 429** = forgot the `[]` on a fetching `useEffect` (infinite loop). **HTTP 401** = missing `X-CS571-ID` header.

07 Visual Design

DESIGN

7 Elements of design	7 Principles of design
Space: positive (subject) / negative (around it)	**Focal point**: where attention goes first
Line: divides space, calls attention, organizes	**Contrast**: very different elements guide attention
Shape: space outlined by a contour	**Balance**: symmetry or asymmetry
Size (scale): relative extent	**Movement**: a flow directing attention
Pattern (repetition): systematic duplication	**Rhythm**: patterned use conveying movement/order
Texture: tactile/visual quality	**Perspective**: sense of horizon and depth
Value: intensity of expression	**Unity**: holistic consistency

- **Glyph** = a character · **typeface** = a design of glyphs (Garamond) · **font** = one instance (Garamond Bold 12pt).
- **Serif** (old-style tapered; modern/slab very thin/thick) · **sans serif** (none, uniform stroke) · **script** (cursive, connected) · **decorative** (evokes a context or feeling).
- **Font parameters:** style (bold, italic, oblique), caps (all/small caps), weight.
- **Leading** = line spacing · **tracking** = overall letter spacing · **kerning** = space between a letter pair · **widows/orphans** = stranded words or lines · **rag** = uneven text edge · **rivers** = white gaps down a paragraph.
- **Color:** the perception of light reflecting from an object; it creates emphasis, organizes, and evokes emotion. **Tint** = hue + white, **shade** = + black, **tone** = + gray. **HSV** = hue, saturation (gray → vivid), value (light → dark).
- **Palettes: analogous** (adjacent hues) · **complementary** (opposite hues) · **split complementary** (a hue + its complement's two neighbors) · **triadic** (three evenly spaced hues) · **monochromatic** (one hue's tints, shades, and tones) · **achromatic** (black, white, grays).
- **Color-blind safe:** emphasize with *intensity* (not hue); mind the *size* of colored elements and the *proximity* of similar colors.

08 Web Dev 2

LIFECYCLE · LISTS · INPUTS

Trigger → Render → Commit → useEffects

- **Trigger** = initial render, state change, or parent re-render (can be batched). **Render** = run the component → virtual DOM. **Commit** = sync to the real DOM (expensive, periodic).
- `useEffect(cb, [])` runs on mount only · `[a]` on mount + when `a` changes · no array: every render. Use it to sync with *external systems*; compute derived values during render instead of in effects.
- `key`: on the parent-most element in a list; unique among siblings; avoid the index.
- **Controlled** input = `value` + `onChange` · **uncontrolled** = `useRef`. Pair the input's `id` with the label's `htmlFor`.

09 Web Design

DESIGN

- **WIMP** = Windows, Icons, Menus, Pointer (the desktop paradigm).
- **Windows** are resizable app containers. *Primary* = main function (canvas). *Secondary* = support via *modal/modeless* operations. They are *overlapping* or *tiled*; secondary windows are *docked*, *stacked*, or *floating*.
- **Menus** list all functions (educational, reference). **Toolbars, palettes, sidebars, tooltips** = frequently used functions. **Tool palettes** = advanced controls. **Pointing** = direct manipulation.
- **Desktop:** complexity in *functionality*. **Web:** complexity in *information structure*; the page is the building block.
- **Primary nav aids** = menus/menubars for major sections. **Secondary** = comprehensive links (fat, left-hand, footer nav). **Breadcrumbs** and

hierarchical lists lead back home.

- **The fold:** what's visible on load vs. below. **Pagination** (discrete pages) vs **infinite scroll** (incremental). **Search** lists content by query; **faceted search** sorts and filters the results.

LAYOUT

- **Layout** = arranging visual elements on a canvas. **Focal point:** the center of visual interest; it locks attention, then guides it onward.
- **Golden ratio** `1:1.618` (asymmetrical but balanced). **Rule of thirds:** a 3×3 grid whose intersections are focal points.
- **Imagery:** on top, not bottom; facing the next focal point; never flipped; don't interrupt headlines or wrap text.
- **Gestalt:** *proximity* (near = grouped) · *similarity* (alike = related) · *continuity* (follow lines) · *closure* (complete shapes).
- **Grids:** an orderly, balanced framework. **Type:** headlines and text blocks guide attention. **Negative space:** a visual break and balance.
- **Visual hierarchy:** position and size show importance. **Scan patterns:** F and Z. **Contrast/emphasis:** position, size, color, type.
- **Design patterns.** Pros: save time, better quality, familiarity, a baseline. Con: may not exist for new design spaces.

10 Web Dev 3

STATE · ROUTING

- **Sharing state:** callbacks (child → parent), context, browser storage, cookies + API, libraries (Redux…). **Prop drilling** = bad practice.
- **Context:** `createContext()` → `<Ctx.Provider value={…}>` → `useContext(Ctx)` in any descendant.
- **Cookies**: usually set by the server (`Set-Cookie`). **sessionStorage**: form data, cleared when the tab closes. **localStorage**: long-lasting data, persists until cleared. All three store **strings** only and **don't re-render**.
- React is a **library, not a framework**. Routers: `Browser` (typical), `Memory` (path hidden), `Hash` (SPA deploys, e.g. GitLab Pages), `Static` (server-side rendering).
- `<Outlet/>` renders the matched child route · `index` = default child · `path="*"` = catch-all.

11 Interaction Design

DESIGN

- **Interaction design:** defining system behaviors that engage the user's full *perception, cognition, and movement*. Unlike visual design, it is tied to behavior and context.
- **5 dimensions:** 1D words · 2D visuals · 3D physical objects/space · 4D time · 5D behavior.
- **Design paradigm:** an archetypal solution or approach to design problems.
  - **Implementation-centric:** mirrors how functions are implemented. Easy to build and debug, but users must learn the functions and can't do high-level actions.
  - **Metaphoric:** a real-world metaphor jump-starts mental models and eliminates learning. A *global* metaphor suits expert interfaces, but doesn't scale, lacks analogs for new capabilities, and varies by culture.
  - **Idiomatic:** dedicated, expressive interactions users *must learn* (e.g. cursor ↔ mouse).
- **Affordance:** perceived properties that hint at how to interact. **True**: implies the action. **False**: perceived but doesn't work. **Hidden**: exists but not obvious.
- **Navigation principles:**
  - **Wayfinding:** signage, environmental clues, maps.
  - **Cost:** the time and effort to navigate; minimize context switches, errors, delays, and steps.
  - **Aids:** global (menus, tabs), utility (sign in, help), associative/in-line (related links).
  - **Models:** see the table.

Nav model	Definition
Hub & spoke	a central hub links to and from specialized components
Fully connected	a central page links to all; all link to each other
Multi-level	main components fully connected; sub-components only among themselves
Stepwise	a sequential/branching process (checkout)
Pyramid	stepwise + jump to the hub and back at each step
Pan-and-zoom	continuous movement over a large space (map, document)
Flat	a central workspace is always visible; no context switch
Modal panel	flat + temporary modal dialogs over the canvas
Clear entry points	direct to frequent functions, skipping the hierarchy
Bookmarks	jump to a chosen point, even deep inside
Escape hatch	back to the main page without retracing steps

12 Web Dev 4

APIS · USEREF · COOKIES

CRUD	HTTP
Create	`POST`
Read	`GET`
Update	`PUT`
Delete	`DELETE`

Status codes (2xx ok · 4xx client · 5xx server)			
200	OK	**401**	Unauthorized
304	Not Modified	**404**	Not Found
400	Bad Request	**409**	Conflict
500	Internal Server Error	**413**	Entity Too Large

- `/players/Rodgers/stats?since=2010`: `Rodgers` is a **path** parameter; `since` is a **query** parameter.
- A JSON body needs `JSON.stringify` + the header `Content-Type: application/json`. **Pre-flight** = an extra `OPTIONS` request (CORS).
- `useRef()` → `{ current }`, which persists across renders; changing it does **not** re-render. Read `ref.current.value` for uncontrolled inputs.
- **JWT:** a signed, short-lived access token used in lieu of a username and password. It is stored in an **HTTP-Only cookie** (set via `Set-Cookie`, unreadable by JS). Requests that set or use cookies need `credentials: "include"`.
- `npm run build` transpiles JSX into a bundle of HTML, CSS, and JS (one `index.html`).

13 Accessibility

DESIGN

- **Usability:** the effectiveness, efficiency, and satisfaction with which specified users achieve specified tasks in a particular environment.
- **Accessibility:** usability by people with the widest range of capabilities (focus: people with disabilities).
- **Disability** has 3 dimensions: *impairment* → *activity limitation* → *participation restrictions*.
- **Impairments:** *sensory* (low vision, blindness, color blindness, deafness) · *physical* (motor/mobility) · *cognitive* (learning, memory, seizures). They vary in severity and structure.
- **Over time:** *permanent* (missing limb) · *temporary* (broken arm) · *situational* (low light, noise).
- **Medical model:** disability is a *personal attribute*. **Social model:** it is *context-dependent* (disabled by societal barriers). **Ability + Context = Disability**.
- Accessibility is achieved through **accessible design** and **assistive technologies**.
- **Universal design:** products and environments usable by all people, to the greatest extent possible, without adaptation or specialized design. What helps some helps all (e.g. closed captions).

UD principle	Key guidelines
1. Equitable use	same means of use for all; don't segregate or stigmatize; appealing to all
2. Flexibility in use	choice of methods; either hand; precision; the user's pace
3. Simple & intuitive	no extra complexity; meets expectations; prompts and feedback
4. Perceptible info	redundant modes (pictorial, verbal, tactile); contrast; legibility
5. Tolerance for error	minimize hazards; warnings; fail-safes
6. Low physical effort	neutral posture; reasonable force; minimal repetition
7. Size & space	line of sight; comfortable reach; room for assistive devices

- **WCAG** levels: **A** (lowest) → **AA** → **AAA**. Level A examples: a label for every form control, no skipped header levels, correctable errors, sufficient color contrast. This is the floor! Test manually and with automated tools (WAVE, AXE).
- **Assistive technologies** are specialized tools that close accessibility gaps: screen readers (JAWS, NVDA, VoiceOver, TalkBack), screen magnifiers, text readers, Braille displays, alternative input (head/mouth wands, eye tracking, sip-and-puff, speech), and AAC (for people who can't use verbal speech).